Multi Robot Planning and Coordination (16-891) Spring 2026 - The Robotics Institute

Homework 2 : Multi-Agent Path Finding (MAPF)

Instructor: Jiaoyang Li TA: Philip Huang, Jingtian Yan

Ranai Srivastav

March 18, 2026

Collaborators:

1. Tom Gao - Discussed problems, brainstormed edge cases
2. Claude - Took help with understanding algorithms, format latex. All code, answers to theory questions, and analysis is written by me.

Task 1 - Included Code

Task 2 - Is there a need to find edge constraints when k $\geq$ 1

No, there is no need to find edge constraints when k is larger than 0.
Let's assume two agents, A and B, where A is at P and B is at Q at time t. By definition, an edge conflict will occur when A tries to go to Q and B tries to go to P at time t+1. Both P and Q are in the trajectories of each of the agents and since A and B can go from P to Q or vice versa in one timestep, P and Q must be neighbors.

Now there are two cases:

1. Agent A waits at current state for some non-zero number of steps, in which case an edge conflict cannot occur, only a vertex constraint. Thus, P and Q cannot be the same state.
2. Agent A has moved at timestep t. For an edge conflict to occur, the next step for both agents must be within reach in one timestep. This is only possible if both agents are in each other's 1-action neigborhood, which implies that any locations farther than the 1-action neighborhood do not need to be checked. In other words, since the robots need to exchange positions along the same edge for an edge conflict, and agents cannot teleport, the states can only be one timestep away, exactly.

Running KR-CBS with k=0 in a realistic setting would mean that we are very sure of each robot's execution and pose at all times. This is an unrealisitic assumption and even if execution is not

an issue, planning a path like this would mean robots will be too close to each other. If in a hierarchical planning framework, this would mean that the local agent-level trajectory follower will have to replan whenever the speed assumptions are violated.

Task 3 - Included Code

Task 4 - Running Statistics

Map Name	CPU Time (s)	Sum of Costs	Num Expanded	Generated Nodes
instances/test_1.txt	0.00	24	4	7
instances/test_1.txt	0.11	28	78	155
instances/test_1.txt	0.01	22	5	9
instances/test_1.txt	0.04	26	29	57
instances/test_1.txt	0.01	26	7	13
instances/test_1.txt	0.01	26	7	13
instances/test_1.txt	0.00	20	4	7
instances/test_1.txt	0.00	24	2	3
instances/test_1.txt	0.11	32	82	163
instances/test_1.txt	0.00	28	2	3
instances/test_1.txt	0.00	18	2	3
instances/test_1.txt	0.01	28	8	15

Table 1: TA-Random results on instances/test_1.txt ($k = 2$, 12 samples)

Stats on sum of costs (12 samples):
Min = 18 Max = 32 Mean = 25.16, Std. Dev. = 3.69, Variance ≈ 13.61

Stats on number of generated nodes (12 samples):
Min = 3 Max = 163 Mean = 37.33, Std. Dev. = 56.18, Variance ≈ 3156.19

Task 5 - Included Code

Task 6 - Stats for TA-Distance

CPU time (s): 0.02 Sum of costs: 16 Expanded nodes: 3 Generated nodes: 5 The sum of costs is smaller for TA-DISTANCE and so is the number of generated nodes. The sum of the lengths of the paths of each agent is smaller than TA-DISTANCE and because TA-DISTANCE uses the hungarian algorithm which gaurantees the optimal solution, this cost is the lowest possible cost achievable in this map given that any of the limitations of hungarian algorithm are not met such as a changing map.

Task 7

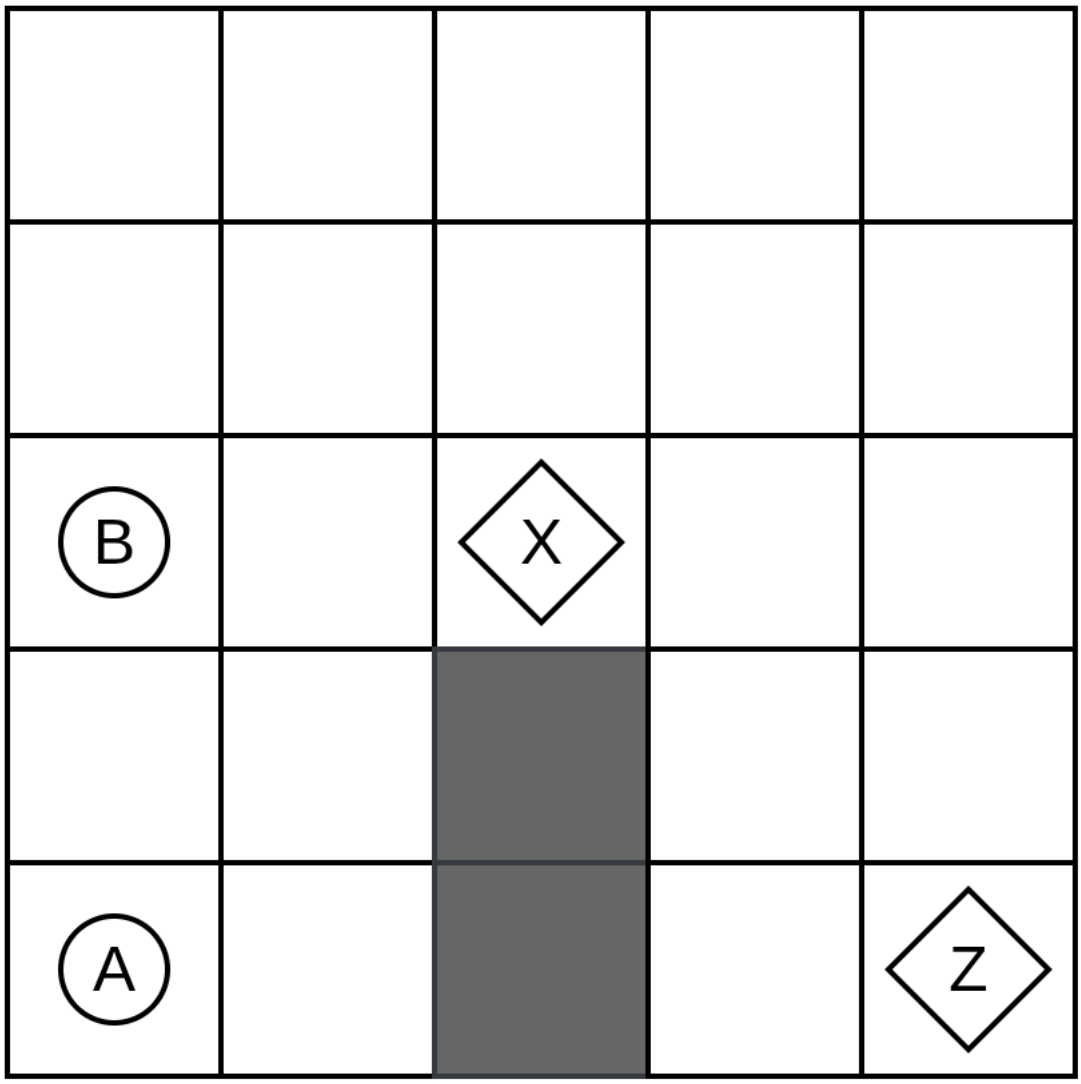

Figure 1: Assigning tasks once cannot take into account changing cost maps because of this task allocation

In the above map, the total distance minimizing allocation also increases the distance for the other agent. Since the Hungarian algorithm assumes a static map, it cannot take a changed map into account because of its allocations and plans a suboptimal path.

Map	TA-RANDOM	TA-DISTANCE	TA-CBS
Test 1	30, 6/11	16, 2/3	16, 2/3
Test 4	31, 36/60	17, 4/6	17, 3/5
Test 8	47, 3118/6004	14, 1/1	14, 1/1

Table 2: (Sum of path lengths), (Num nodes explored)/(Num nodes expanded)
All maps except Map 8 using TA-RANDOM were solved in 0.0s, whereas Map 8 took 10.08s. The assignment using TA-CBS is optimal in terms of number of nodes expanded and the path cost. Since the number of nodes expanded is smaller, the runtime also finished faster! In this case, the difference between TA-DISTANCE and TA-CBS is negligible but in cases when the limtations of the hungarian algorithm are met, such as if the map is dynamic in nature or a planned allocation ends up changing the map in some way. TA-CBS will be able to deal with these challenges better.

Task 8

This is not possible if all agents can reach all goals at the start and all edges are bi-directional. The Hungarian algorithm is optimal and complete for a bipartite graph given that an assignment does not cause a change in the graph and no mistakes are made in assigning agents (quadruped gets assigned aerial goal). If the costs change because of the assignment, the hungarian algorithm does not take this into account. But if the only thing is that the map changes, if a path exists, CBS will find it, even if it is HIGHLY suboptimal. If the robot traverses a path that is 1 directional - it can go through a turnstile only in one direction, then in that case, the robot will not be able to come back and an example can be constructed where the agent gets assigned a task it cannot reach.

In a trivial 1D case, the cost minimizing distance is always one that maintains the order.

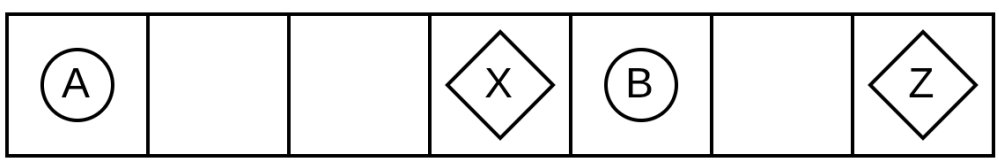

Figure 2: I originally thought this was impossible but it is not

In more complex cases, CBS will usually be able to find a path as long as all nodes are reachable from all nodes and costs are computed optimally.

Task 9 - Implemented in code

Task 10 -

Task 11

Since the input paths for the execution come from a CBS run, we are guaranteed that paths are collision free unless a robot has a malfunction. In a general MAPF setting, assume 2-3 robots share the same path but separated by a sufficient distance k, where a robot visiting a location before another created a predecessor-successor relation. In this case, even if this path is not a corridor, successive agents might still fail to plan a path successfully around a failed predecessor robot. They might continue to wait for a robot with a failure, assuming that it will move but it might not.

Task 12

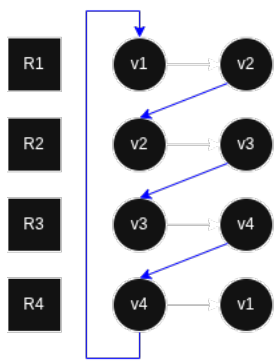

Figure 3: For each agent R[1-4], the circles denote which vertex they will be at. The vertex on the left denotes current time and the vertex on the right is the next location.

Here we can see that the TPG is cyclic and thus, when attempting to resolve this graph into a sequence of actions for each agent, we will not be able to decompose it. Because the TPG has a cycle, it also is a good example of a deadlock!

Map	Path-Cost
Exp_20	33, 358/666 in 2.27s
Exp_21	57, 793/1442 in 8.39s
Exp_22	47, 18/35 in 0.14s
Exp0	7, 2/2 in 0.0s
Exp_2_2	7, 3/5 in 0.0s

Table 3

Task 13

Task 14

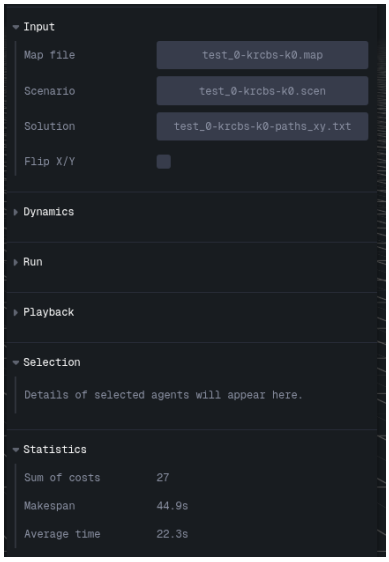

Figure 4: Output of SMART on Test0 with k=0

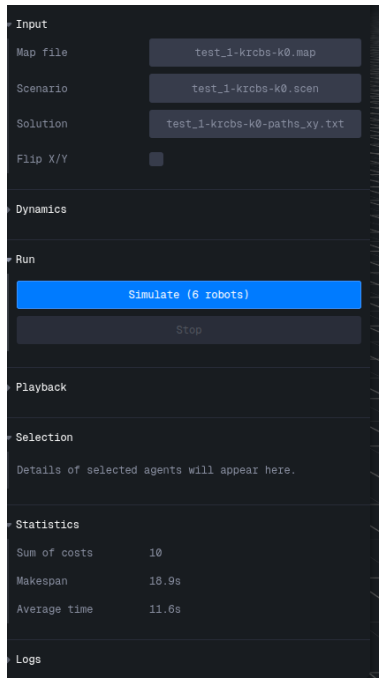

Figure 5: Output of SMART for Test1 with k=0

Task 15

The cost reported by the `sum_of_costs` parameter is the sum of the actions taken by each of the agents. The action space for both the simulation environments is different - while we have a holonomic robot that can move freely in any direction, the robot in SMART can only rotate in place and move straight. This means that to go from point A to B where A is at the bottom of a grid and B is at the top, a holonomic robot will have nearly half the cost of a non holonomic robot like the one in SMART. Thus, the costs will differ.

Since turning is now an action, this means that the robot's position does not change for 2 timesteps every time it rotates, meaning the k-robust planning might have to wait for longer or plan for longer to move around other robots, especially if in a bottleneck. If the robot has a p probability of failure during execution, this implies it can be stuck along its path with a higher probability.

All of this follows from the fact that the path is longer now.

Task 16

```
def a_star(my_map, start_loc, goal_loc, h_values, agent, constraints, max_path_length=-1)
    :
    """ my_map      - binary obstacle map
        start_loc   - start position
        goal_loc    - goal position
        h_values    - precomputed heuristic values for each location on the map
        agent       - the agent that is being re-planned
```

```
    constraints - constraints defining where robot should or cannot go at each
timestep
"""

    ############################
    # Extend the A* search to search in the space-time domain
    #           rather than space domain, only.

    open_list = []
    closed_list = dict()
    earliest_goal_timestep = 0
    prev_goal_timestep = 0
    h_value = h_values[start_loc]
    agent_table = build_constraint_table(constraints, agent)

    # If the start position is forbidden at t=0, no path exists
    if is_constrained(start_loc, start_loc, 0, agent_table):
        return None, float('inf')
    root = {'loc': start_loc,
            'g_val': 0,
            'h_val': h_value,
            'parent': None,
            "timestep": 0}

    push_node(open_list, root)
    closed_list[(root['loc'], root["timestep"])] = root

    while len(open_list) > 0:
        curr = pop_node(open_list)

        if max_path_length != -1 and curr["timestep"] > max_path_length:
            return None, float('inf')

        ############################
        # Task 2.2: Adjust the goal test condition to handle goal constraints
        if curr['loc'] == goal_loc:
            prev_goal_timestep = curr["timestep"]
            can_stop = True
            for k in agent_table.keys():
                if k > prev_goal_timestep:
                    can_stop = False
                    break

            if can_stop:
                return get_path(curr), curr["g_val"]  ←

        for dir in range(len(directions)):
            child_loc = move(curr['loc'], dir)
            if (not in_map(my_map, child_loc) or
                    my_map[child_loc[0]][child_loc[1]] or
                    is_constrained(curr["loc"], child_loc, curr["timestep"] + 1,
                                   agent_table)):
                continue
```

```
                child = {'loc': child_loc,
                         'g_val': curr['g_val'] + 1,
                         'h_val': h_values[child_loc],
                         'parent': curr,
                         "timestep": curr["timestep"] + 1}

                ### To match the costs of the SMART simulator!
                child["g_val"] = increase_cost_2(child)
                ##############################################

                if (child['loc'], child["timestep"]) in closed_list:
                    existing_node = closed_list[(child['loc'], child["timestep"])]
                    if compare_nodes(child, existing_node):
                        closed_list[(child['loc'], child["timestep"])] = child
                        push_node(open_list, child)
                else:
                    closed_list[(child['loc'], child["timestep"])] = child
                    push_node(open_list, child)

        return None, float('inf')
```

```
def increase_cost_2(node: Dict) -> int:
    cost_increase = 0
    if node['g_val'] > 1:
        prev_prev_loc = node["parent"]["parent"]["loc"]
        prev_loc = node["parent"]["loc"]
        curr_loc = node["loc"]

        p2p_action = (prev_prev_loc[0] - prev_loc[0], prev_prev_loc[1] - prev_loc[1])
        p2c_action = (prev_loc[0] - curr_loc[0], prev_loc[1] - curr_loc[1])

        if p2p_action != p2c_action:
            cost_increase += 1

    return node["g_val"] + cost_increase
```

Task 17